

ChipChat Assignment Report

Overview

In this assignment, I used the ChipChat workflow to generate Verilog from natural-language prompts and then checked the results with Icarus Verilog. I completed the eight examples in the notebook and also added the 8-bit CPU example. Most examples did not work perfectly on the first try, so I used the compiler or testbench output as feedback and regenerated the code until the tests passed.

I used Google Colab for the notebook. The OpenAI API key was stored in Colab Secrets, so the key was not written directly in the notebook. GPT-5.6 was used for most of the later work and for the CPU design.

Eight Notebook Examples

#	Example	What it does	Main issue / change	Result
1	Binary-to-BCD	Convert a 5-bit binary input to BCD.	The first code compiled but failed a test. The double-dabble order was corrected after feedback.	PASS
2	Sequence Detector	Detect 001, 101, 110, 000, 110, 110, 011, 101.	The first result asserted sequence_found at the wrong time. The output timing was changed.	PASS
3	Shift Register	8-bit serial-in shift register with enable and active-low reset.	There were reset/interface issues when matching the provided testbench. The design was regenerated with the testbench requirements.	PASS
4	ABRO State Machine	One-hot ABRO state machine with current State output.	The generated module/port names did not match the testbench at first. The interface was corrected.	PASS
5	Dice Roller	Generate dice results for the required selectable dice sizes.	One API response did not return usable Verilog, so I regenerated it with the testbench included.	PASS
6	LFSR	8-bit LFSR with the required initial value and taps.	The first module name did not match the testbench. The final version was regenerated and verified.	PASS
7	Sequence Generator	Generate and repeat the required fixed byte sequence.	This example matched the testbench well and passed without much debugging.	PASS
8	Traffic Light	Traffic-light FSM with red, green, and yellow timing.	The first top-module name was different from the testbench. After matching the interface, the test passed.	PASS

8-bit CPU Example

For the CPU part, I did not ask the model to generate the whole processor immediately. I first used the starting prompt from the assignment and asked how to begin a very small 8-bit accumulator-based processor under a tight area and I/O constraint.

Starting prompt

Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator-based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?

Architecture I used

- 8-bit accumulator-based CPU with an 8-bit program counter.
- Registers: A, X, PC, IR, plus Z and C flags.
- One-byte instructions. The upper 4 bits are the opcode and the lower 4 bits are used as an immediate or branch offset when needed.
- Unified 256-byte external instruction/data memory.
- Three control states: FETCH, EXECUTE, and HALT.
- No interrupts and no hardware stack.
- Synchronous active-low reset.

Instruction set

Opcode	Instruction	Opcode	Instruction
0x0	NOP	0x1	LDI4 imm4
0x2	ORI4 imm4	0x3	TAX
0x4	TXA	0x5	LDA [X]
0x6	STA [X]	0x7	ADD [X]
0x8	SUB [X]	0x9	INX
0xA	DEX	0xB	BRA rel4
0xC	BZ rel4	0xD	BNZ rel4
0xE	JMP X	0xF	HALT

CPU verification

I generated the final cpu8 Verilog module and then wrote a testbench with a 256-byte memory. Instead of only checking one instruction at a time, I loaded two small machine-code programs and let the CPU execute them. The programs exercised arithmetic, load/store, branches, JMP, flags, index-register operations, and HALT.

```
PROGRAM 1 PASSED
PROGRAM 2 PASSED
ALL CPU TESTS PASSED!
```

Human Feedback and Debugging

The main thing I noticed is that the LLM could usually generate code that looked reasonable, but small details still caused failures. The most useful feedback was very specific. For example, in the binary-to-BCD example, just saying that the answer was wrong did not help much. After I pointed out the order of the double-dabble operations, the next version passed. In the sequence detector, the important issue was that `sequence_found` needed to go high on the same cycle as the last input.

Several failures were not complicated logic bugs. They were simple integration problems such as the module name or reset signal name being different from the provided testbench. After this happened a few times, I included the testbench interface in the prompt so the generated module would match it exactly. This made the later examples easier to finish.

The CPU benefited more from conversation than the smaller examples. Before generating Verilog, I first fixed the CPU registers, instruction format, memory interface, flags, and FSM states. This reduced ambiguity and made the generated CPU easier to test.

Strengths and Limitations

Strengths:

- It can produce a useful RTL starting point very quickly.
- Giving simulator errors back to the model is an efficient way to debug many small designs.
- For a bigger design such as the CPU, the model is useful for discussing architecture before writing code.

Limitations:

- Generated code can look correct but still fail because of timing or algorithm details.
- The model may choose different module or port names unless the interface is stated clearly.
- Passing the provided testbench does not prove that every possible corner case or synthesis target is correct.

Conclusion

I completed all eight notebook examples and the additional 8-bit CPU example. All of the final versions passed their simulations. This assignment showed me that using an LLM for hardware design is not just asking for Verilog once. The useful part is the loop of generating code, running the testbench, looking at the exact failure, and giving that information back to the model. For the CPU, making the architectural decisions before asking for RTL was also important.